



Securing Apache

Part—7

Moving deeper into Web application and Apache security, let's now focus on OS commanding attacks, and those that lead to the disclosure of crucial information and server directory paths. The attacks described below are often overlooked or trivialised.

OS commanding (a.k.a. OS command execution, shell injection) is a vulnerability in websites that lets attackers execute operating system commands through the manipulation of application input. It occurs when scripts execute external commands that are constructed using unsanitised input data. Such commands will run with the privileges of the component (database server, Web application server, Web server, wrapper or application) that executed the command. This lets the attacker gain access to otherwise unreachable areas (e.g., operating system directories and files), inject unexpected and dangerous commands, upload malicious programs or even obtain passwords directly from the operating system. Figure 1 visualises such an attack. OS commanding vulnerabilities are frequently found in Perl and PHP scripts, because these programming environments encourage programmers to reuse operating system binaries. However, there are also chances of such vulnerabilities in Java and ASP. Let's take a look at a few Perl and PHP examples.

The following Perl CGI code is part of a Web application for server administration. This function allows administrators to specify a directory on the server, and view a summary of its disk usage:

```
#!/usr/bin/perl
use strict;
use CGI qw(:standard escapeHTML);
print header, start_html("");
print "<pre>";
my $command = "du -h --exclude php* /var/www/html";
$command= $command.param("dir");
$command=$command;
print "$command\n";
print end_html;
```

When used as intended, this script simply appends the value of the user-supplied *dir* parameter to the end of a specific command, which it executes, and displays the results. A normal usage URL might be something like <http://www.example.com/cgi-bin/showinfo.pl?dir=/public>.

This functionality can be exploited in various ways, by supplying crafted input containing shell characters that have a special meaning to the command interpreter. For example, the pipe character (`|`) is used to redirect output from one process to the input of another, enabling the chaining of multiple commands. An attacker can leverage this to inject a second command (and retrieve its output) with a malicious URL such as <http://www.example.com/cgi-bin/showinfo.pl?dir=/public;%20cat&%20/etc/passwd>. Here, the original *du* command's output is redirected as input to the *cat /etc/passwd* command, which simply ignores it, and just outputs the contents of the *passwd* file.



Note: Most operating systems restrict access to system files. Also, the advent of shadow passwords has rendered this specific attack less useful than it previously was. Still, damaging attacks can be made by obtaining *.php* files which may contain database host-names, user-names and passwords, or other configuration data, or by obtaining the database files themselves.

PHP also provides functions, such as *passthru()* and *exec()*, and the back-tick operator, to execute external programs. Consider the following PHP code, which displays a list of files in a particular user's home folder:

```
$output = `ls -al /home/$username`;
echo $output;
```